

Complete SQL Course with PostgreSQL & MySQL

PRACTICAL GUIDE USING PGADMIN AND XAMPP

PRESENTED BY: UJJWAL NEUPANE



✿ SQL SELECT STATEMENTS

SELECT * FROM tbl

Select all rows and columns from table tbl

SELECT c1,c2 FROM tbl

Select column c1, c2 and all rows from table tbl

SELECT c1,c2 FROM tbl

WHERE conditions

ORDER BY c1 ASC, c2 DESC

Select columns c1, c2 with where conditions and from table tbl order result by column c1 in ascending order and c2 in descending order

**SELECT DISTINCT c1, c2
FROM tbl**

Select distinct rows by columns c1 and c2 from table tbl.

**SELECT c1, aggregate(expr)
FROM tbl
GROUP BY c1**

Select column c1 and use aggregate function on expression expr, group columns by column c1.

**SELECT c1, aggregate(expr) AS c2
FROM tbl
GROUP BY c1
HAVING c2 > v**

Select column c1 and c2 as column alias of the result of aggregate function on expr. Filter group of records with c2 greater than value v

✿ SQL UPDATE TABLE

**INSERT INTO tbl(c1,c2,...)
VALUES(v1,v2...)**

Insert data into table tbl

**INSERT INTO tbl(c1,c2,...)
SELECT c1,c2.. FROM tbl2
WHERE conditions**

Insert data from tbl2 into tbl

**UPDATE t
SET c1 = v1, c2 = v2...
WHERE conditions**

Update data in table tbl

**DELETE FROM tbl
WHERE conditions**

Delete records from table tbl based on WHERE conditions.

TRUNCATE TABLE tbl
Drop table tbl and re-create it, all data is lost

✿ SQL TABLE STATEMENTS

**CREATE TABLE tbl(
 c1 datatype(length)
 c2 datatype(length)
 ...
 PRIMARY KEY(c1)
)**

Create table tbl with primary key is c1

DROP TABLE tbl

Remove table tbl from database.

ALTER TABLE tbl

ADD COLUMN c1 datatype(length)

Add column c1 to table tbl

ALTER TABLE tbl

DROP COLUMN c1

Drop column c1 from table tbl

✿ SQL JOIN STATEMENTS

SELECT * FROM tbl1

INNER JOIN tbl2 ON join-conditions

Inner join table tbl1 with tbl2 based on join-conditions.

SELECT * FROM tbl1

LEFT JOIN tbl2 ON join-conditions

Left join table tbl1 with tbl2 based on join-conditions.

SELECT * FROM tbl1

RIGHT JOIN tbl2 ON join-conditions

Right join table tbl1 with tbl2 based on join-conditions.

SELECT * FROM tbl1

RIGHT JOIN tbl2 ON join-conditions

Full outer join table tbl1 with tbl2 based on join-conditions.

What is SQL?

- SQL = Structured Query Language
- Manage data in relational databases
- Database is a system where the users will store and manage data.
- SQL Categories:
 - DDL(Data Definition Language): CREATE, DROP, ALTER
 - DML(Data Manipulation Language): SELECT, INSERT, UPDATE, DELETE
 - DCL(Data Control Language): GRANT, REVOKE
 - TCL(Transaction Control Language): COMMIT, ROLLBACK

Types of DataBases

Feature	Relational Database (RDBMS)	Non-Relational Database (NoSQL)
Data Model	Stores data in tables (rows & columns)	Stores data in various formats: documents, key-value pairs, graphs, or wide-columns
Schema	Fixed schema → must define tables & columns before inserting data	Flexible schema → data can be inserted without predefined structure
Relations	Uses relationships (foreign keys, joins) between tables	Usually no joins , relationships handled at the application level
Scalability	Vertical scaling (add more CPU/RAM to one server)	Horizontal scaling (add more servers to handle large data)
Best for	Structured data, complex queries, financial systems, ERP, CRM	Unstructured/semi-structured data, real-time apps, IoT, big data
Examples	MySQL, PostgreSQL, Oracle, MS SQL Server	MongoDB, Cassandra, CouchDB, Redis, Neo4j

1. DDL (Data Definition Language)

Purpose: Defines the structure of database objects such as tables, schemas, and indexes.

Keyword	Description
---------	-------------

CREATE	Creates new tables, databases, views, or other objects.
--------	---

Example: CREATE TABLE students (id INT, name VARCHAR(50));

DROP	Deletes tables, databases, or other objects.
------	--

Example: DROP TABLE students;

ALTER	Modifies an existing object (e.g., add or delete a column).
-------	---

Example: ALTER TABLE students ADD age INT;

2. DML (Data Manipulation Language)

Purpose: Works with data stored in the database (insert, retrieve, update, delete).

Keyword	Description
---------	-------------

SELECT	Retrieves data from the database.
--------	-----------------------------------

Example: SELECT * FROM students;

INSERT	Adds new data into a table.
--------	-----------------------------

Example: INSERT INTO students (id, name) VALUES (1, 'John');

UPDATE	Modifies existing data.
--------	-------------------------

Example: UPDATE students SET name = 'Alice' WHERE id = 1;

DELETE	Removes data from a table.
--------	----------------------------

Example: DELETE FROM students WHERE id = 1;

3. DCL (Data Control Language)

Keyword	Description
GRANT	Gives user access rights to the database. <i>Example:</i> GRANT SELECT ON students TO user_name;
REVOKE	Removes access rights given by the GRANT command. <i>Example:</i> REVOKE SELECT ON students FROM user_name;

4. TCL (Transaction Control Language)

Keyword	Description
COMMIT	Saves all the changes made during the current transaction. <i>Example:</i> COMMIT;
ROLLBACK	Undoes changes made in the current transaction. <i>Example:</i> ROLLBACK;

PostgreSQL vs MySQL

PostgreSQL | MySQL

-----|-----

Object-Relational | Relational

Advanced JSON | Basic JSON

Better for complex queries | Faster for reads

Tool: pgAdmin | Tool: phpMyAdmin

Syntax differences exist

A **Relational Database Management System (RDBMS)** stores data in **tables** with rows and columns. These tables can be related using **foreign keys**, and they follow **relational algebra**.

An **Object-Relational Database Management System (ORDBMS)** is an advanced form of RDBMS that supports **object-oriented features**

Installing PostgreSQL

1. Download from [postgresql.org](https://www.postgresql.org)
2. Install PostgreSQL & pgAdmin
3. Default port: 5432
4. Use pgAdmin to create servers and databases

Installing MySQL with XAMPP

1. Download XAMPP from apachefriends.org
2. Start MySQL in XAMPP Control Panel
3. Access via phpMyAdmin or CLI
4. Default port: 3306

Writing Clean SQL Codes

Writing clean SQL code is essential for readability and maintainability. The following tips apply to SQL in general, not just SQL SELECT statements:

- **Use Capital Letters for Keywords:** SQL is not case sensitive. However, it is better to use capital letters for keywords such as SELECT and FROM to make the query more readable.
- **Use Proper Spacing:** While it is possible to write SQL queries on a single line, using proper indentation and whitespace greatly enhances readability.
- **Use Comments for Long Queries:** Comments are helpful descriptions that explain what the SQL code is doing. They are ignored by the database system during execution. In SQL, any text between -- and the end of the line is treated as a comment. Use /* ... */ for multiline comments.

Creating a Database

-- SQL

```
CREATE DATABASE school;
```

pgAdmin: Right click → Create → Database

phpMyAdmin: Click New → Enter name → Create

Creating Tables

```
CREATE TABLE students (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    age INT,  
    email VARCHAR(100) UNIQUE  
);
```

MySQL uses AUTO_INCREMENT instead of SERIAL

Creating Tables

Full General Syntax:

```
CREATE TABLE table_name(  
column_name TYPE column_constraint  
column_name TYPE column_constraint  
table_constraint table_constraint  
) INHERITS existing_table_name;
```

Common Simple Syntax:

```
CREATE TABLE table_name »  
column_name TYPE column_constraint,  
column_name TYPE column_constraint,  
);
```

Inserting Data

```
INSERT INTO students (name, age, email)
```

```
VALUES ('John Doe', 20, 'john@example.com');
```

Selecting Data

```
SELECT * FROM students;
```

```
SELECT name FROM students WHERE age > 18;
```

Syntax for SELECT statement:

```
SELECT column_name
```

```
FROM table_name
```

Selecting Data

Challenge

Q) We want to send out a promotional email to our existing customers!

Hint: For solving this challenge we need to know name and email of the customers

Use a SELECT statement to grab the first and last names of every customer and their email address.

Use the customer table.

You can use the table drop-down to view what columns are available.

You could also use `SELECT * FROM customer` to see all the columns.

Selecting Distinct Data

SELECT DISTINCT Statement

Sometimes a table contains a column that has duplicate values, and you may find yourself in a

situation where you only want to list the unique/distinct values.

The DISTINCT keyword can be used to return only the distinct values in a column.

The DISTINCT keyword operates on a column.

Selecting Distinct Data

Syntax:

```
SELECT DISTINCT column
```

```
FROM fable;
```

Or, you can also use small bracket as

```
SELECT DISTINCT (column)
```

```
FROM table;
```

Selecting Distinct Data

Challenges

Q) Use what we have learned about DISTINCT to retrieve the different rating types our films could have in our database.

Hint:

Use the film table.

You can use the table drop-down to view what columns are available.

You could also use `SELECT * FROM films` to see all the columns.

COUNT Function

The COUNT function returns the number of input rows that match a specific condition of a

query.

We can apply COUNT on a specific column or just pass COUNT (*).

Syntax:

```
SELECT COUNT(Name)
```

```
FROM table; Or,
```

```
SELECT COUNT(Choice)
```

```
FROM table; Or,
```

```
SELECT COUNT(*)
```

```
FROM table;
```

This is simply returning the number of rows in the table.

WHERE Function

SELECT and WHERE the most fundamental SQL statements.

The WHERE statement allows us to specify conditions on columns for the rows to be returned.

The WHERE clause appears immediately after the FROM clause of the SELECT statement.

The conditions are used to filter the rows returned from the SELECT statement.

Syntax:

```
SELECT column1, column2
```

```
FROM table
```

```
WHERE conditions;
```

Comparison Operators

Operator	Description
=	Equal
>	Greater than
<	Less Than
>=	Greater than or equal to
<=	Less than or equal to
<> or !=	Not equal to

Logical Operators

- Allow us to combine multiple comparison operators
 - **AND**
 - **OR**
 - **NOT**

WHERE Function

Challenge

Q) What is the email for the customer with the name Nancy Thomas?

Hints:

Use the customer table.

Make sure the capitalization and spelling of the names is correct.

Use AND to combine conditions.

Use single quotes around the 'string'.

ORDER BY

PostgreSQL sometimes returns the same request query results in a different order.

We can use ORDER BY to sort rows based on a column value, in either ascending or descending

Pullup or precise seek order.

Order will be alphabetical for string based columns and numerical order for

Order by will be towards the bottom of a query and one of the very last thing

Syntax:

```
SELECT column_1, column_2
```

```
FROM table
```

```
ORDER BY column_1 ASC/DESC
```

LIMIT

LIMIT

LIMIT command allows us to limit the number of rows returned for a query.

It is very useful if we don't want to return every single row in a table, but only view the top few rows to get an idea of the table layout.

LIMIT also becomes useful in combination with ORDER BY.

LIMIT goes at the very end of the query request and is the last command to be executed.

LIMIT

```
SELECT *  
FROM payment  
ORDER BY payment_date DESC  
LIMIT 10;
```

What are the customer ids of the first 10 customers who created a payment?

Hints:

Use the payment table.

We need to use both ORDER BY and LIMIT.

Remember we may need to specify ASC or DESC.

BETWEEN

The BETWEEN operator can be used to match a value against a range of values:

value BETWEEN low AND high. BETWEEN is inclusive and NOT BETWEEN

The BETWEEN operator is same as:

value $\geq$ low AND value $\leq$ high

- value BETWEEN low AND high

We can also combine BETWEEN with the NOT logical operator:

- value NOT BETWEEN low AND high

The NOT BETWEEN operator is the same as:

value $<$ low OR value $>$ high

- value NOT BETWEEN low AND high

BETWEEN

The NOT BETWEEN operator is the same as:

- `value < low OR value > high`

`value NOT BETWEEN low AND high`

The BETWEEN operator can also be used with dates.

We convert 2007-01-26 into the ISO 8601 standard format, which is YYYY-MM-DD

`date BETWEEN '2007-01-01' AND '2007-02-01'.`

IN

We can use the IN operator to create a condition that checks to see if a value is included in a list of multiple options.

Syntax: `value IN(option1,option2,...,option_n)`

LIKE and ILIKE

LIKE is used for pattern matching in strings. It is case-sensitive in PostgreSQL.

% → matches zero or more characters.

Example: '%cat' → ends with "cat".

Example: 'cat%' → starts with "cat".

ILIKE = case-insensitive (PostgreSQL only).

Example: '%cat%' → contains "cat".

_ → matches exactly one character.

Example: 'c_t' → matches "cat", "cut", "cot", but not "cart".

Update

UPDATE students SET age = 21 WHERE id = 1;

UPDATE keyword allows for the changing of the columns in a table.

General Syntax:

UPDATE table_name

SET column1 = value 1,

column2=value2,...

WHERE

condition;

DELETE

DELETE FROM table_name

WHERE condition;

DELETE All Row

DELETE FROM students;

If you forgot WHERE, all rows will be deleted

AS

AS clause allows us to create an "alias" for a column or result.

Syntax:

SELECT column

AS new_name

FROM table;

Constraints

- **PRIMARY KEY** - A PRIMARY KEY uniquely identifies each row in a table. It must be unique and NOT NULL.

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(100)  
);
```

- **UNIQUE** - The UNIQUE constraint ensures all values in a column are different.

```
CREATE TABLE users (  
    user_id INT PRIMARY KEY,  
    email VARCHAR(100) UNIQUE  
);
```

Constraints

- **NOT NULL**- NOT NULL ensures the column cannot be left empty.
-

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL  
);
```

- **Foreign Key** - A FOREIGN KEY connects a record to another table.

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    student_id INT,  
    FOREIGN KEY (student_id) REFERENCES students(student_id)  
);
```

Constraints

- **CHECK**- CHECK is used to validate a condition on a column.
-

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    age INT CHECK (age >= 18)  
);
```

JOINS

Used to combine rows from 2 or more tables.

```
SELECT students.name, courses.name
```

```
FROM students
```

```
JOIN courses ON students.id = courses.student_id;
```

Let's say we have two tables:

student_id	name
1	Ujjwal
2	ABC
3	DEF

student_id	subject	marks
1	Math	90
2	Science	88
4	English	92

Inner Join

It shows only students **who have marks**.

```
SELECT Students.name, Marks.subject, Marks.marks
```

```
FROM Students
```

```
INNER JOIN Marks ON Students.student_id = Marks.student_id;
```

name	subject	marks
Ujjwal	Math	90
ABC	Science	88

Left Join

Shows **all students** even if they have **no marks**.

```
SELECT Students.name, Marks.subject, Marks.marks
```

```
FROM Students
```

```
LEFT JOIN Marks ON Students.student_id = Marks.student_id;
```

name	subject	marks
Ujjwal	Math	90
ABC	Science	88
DEF	NULL	NULL

Right Join

Shows **all marks**, even if **student doesn't exist**.

```
SELECT Students.name, Marks.subject, Marks.marks
```

```
FROM Students
```

```
RIGHT JOIN Marks ON Students.student_id = Marks.student_id;
```

name	subject	marks
Ujjwal	Math	90
ABC	Science	88
NULL	English	92

Outer Join

Shows **all students and all marks**, even if they don't match.

```
SELECT Students.name, Marks.subject, Marks.marks
```

```
FROM Students
```

```
FULL OUTER JOIN Marks ON Students.student_id = Marks.student_id;
```

name	subject	marks
Ujjwal	Math	90
Rojina	Science	88
Anil	NULL	NULL
NULL	English	92

Summary

JOIN Type

What it Shows

INNER JOIN

Matching data from both tables

LEFT JOIN

All from left + matched from right

RIGHT JOIN

All from right + matched from left

FULL OUTER JOIN

All from both tables, matched or not

Aggregate Functions

```
SELECT COUNT(*) FROM students;
```

```
SELECT AVG(marks) FROM students;
```

```
SELECT MAX(marks), MIN(marks) FROM students;
```

```
SELECT SUM(marks) FROM students;
```

Syntax Differences

- AUTO_INCREMENT (MySQL) vs SERIAL (PostgreSQL)
- JOIN and LIMIT differences
- IFNULL vs COALESCE
- Date functions vary

Best Practices

- Use proper data types
- Normalize your tables
- Avoid SELECT *
- Index where needed
- Use transactions